

PREMIUM CONTRACT: ПЕРЕВОДЫ (PAYMENT)

Версия: 1.0.0

Дата: 2026-06-14

Статус: ☒ Complete

Доступ:  Только "Шейх"

1. ОБЗОР

1.1 Назначение

Переводы (Payment) — это premium вложение для мгновенных денежных переводов между пользователями Balloo Messenger напрямую в чате.

1.2 Статус доступа

Параметр	Значение
Доступ	 Premium (Шейх)
Категория	Финансы
Сложность	 Очень высокая
Время реализации	10-14 дней
Юридические требования	Да

1.3 Use Cases

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

 Раздел счёта

"Обед на 3000₽ → разделить на 3 = 1000₽ каждому"

 Возврат долга

"Возвращаю 5000₽ за билет"

 Оплата услуг

"Оплата дизайна логотипа — 15000₽"

 Донаты

"Спасибо за контент!  500₽"

 Аренда

"Аренда квартиры за май — 50000₽"

2. ТИПЫ ДАННЫХ

2.1 Основной интерфейс

```
/**
 * Типы вложений для переводов
 */
export type PaymentAttachmentType = 'payment';

/**
 * Статусы перевода
 */
export type PaymentStatus =
  | 'pending'      // Ожидает подтверждения
  | 'processing'   // В обработке
  | 'completed'    // Успешно завершён
  | 'failed'       // Ошибка
  | 'refunded'     // Возвращён
  | 'cancelled';   // Отменён

/**
 * Методы оплаты
 */
export type PaymentMethod =
  | 'balance'      // Внутренний баланс Balloo
  | 'card'         // Банковская карта
  | 'sbp'          // Система Быстрых Платежей
  | 'yandex'       // Яндекс.Деньги
  | 'tinkoff';     // Тинькофф Касса

/**
 * Валюты
 */
export type PaymentCurrency =
  | 'RUB'
  | 'USD'
  | 'EUR'
  | 'KZT'
  | 'BYN'
  | 'UAH'
  | 'GEL'
  | 'TRY';

/**
 * Вложение перевода
 */
```

```

export interface PaymentAttachment {
  type: 'payment';
  attachmentId: string;
  paymentId: string;

  // Сумма
  amount: number;           // Сумма в копейках/центах
  currency: PaymentCurrency;
  originalAmount?: number;   // Оригинальная сумма (для конвертации)
  originalCurrency?: PaymentCurrency;
  exchangeRate?: number;     // Курс конвертации

  // Стороны
  sender: PaymentUser;
  recipient: PaymentUser;

  // Статус
  status: PaymentStatus;
  statusHistory: PaymentStatusEntry[];

  // Метод оплаты
  paymentMethod: PaymentMethod;
  paymentDetails?: PaymentMethodDetails;

  // Детали
  message?: string;         // Комментарий к переводу
  reference?: string;       // Уникальный референс
  invoiceId?: string;       // ID счёта (если есть)

  // Комиссии
  fee: number;              // Комиссия в копейках
  feePercent: number;       // Процент комиссии
  feePayer: 'sender' | 'recipient' | 'platform';
  sheikhDiscount: boolean;  // Скидка для Шейх (0%)

  // Конвертация
  conversion?: {
    fromCurrency: PaymentCurrency;
    toCurrency: PaymentCurrency;
    rate: number;
    fee: number;
  };

  // Временные метки
  createdAt: number;        // timestamp
  processedAt?: number;
  completedAt?: number;
  expiresAt?: number;       // Когда истекает (для pending)

  // Чек
  receipt?: PaymentReceipt;

  // Безопасность

```

```

security: PaymentSecurity;

// Метаданные
metadata?: Record<string, any>;

// Юридическая информация
legal: PaymentLegalInfo;
}

/**
 * Пользователь в переводе
 */
export interface PaymentUser {
  userId: string;
  displayName: string;
  avatar?: string;
  accountNumber?: string; // Счёт в системе
  phone?: string; // Для СБП
  email?: string;
  isVerified: boolean;
  isSheikh: boolean;
}

/**
 * История статусов
 */
export interface PaymentStatusEntry {
  status: PaymentStatus;
  timestamp: number;
  reason?: string;
  actor?: string; // userId кто изменил
  systemNote?: string;
}

/**
 * Детали метода оплаты
 */
export interface PaymentMethodDetails {
  // Для карт
  cardLast4?: string;
  cardBrand?: 'visa' | 'mastercard' | 'mir' | 'amex';
  cardExpiry?: string; // MM/YY

  // Для СБП
  sbpBank?: string;
  sbpAccountId?: string;

  // Для Яндекс
  yandexWallet?: string;

  // Для Тинькофф
  tinkoffAccountId?: string;
}

```

```

    // Для баланса
    balanceTransactionId?: string;
}

/**
 * Чек
 */
export interface PaymentReceipt {
    receiptId: string;
    receiptUrl: string;
    receiptPdfUrl?: string;
    fiscalNumber?: string;          // Фискальный номер
    fiscalDate?: number;
    items: ReceiptItem[];
    total: number;
    tax: number;
    paymentType: 'income' | 'outcome';
}

export interface ReceiptItem {
    name: string;
    quantity: number;
    price: number;
    total: number;
    taxRate: number;
    tax: number;
}

/**
 * Безопасность
 */
export interface PaymentSecurity {
    encrypted: boolean;
    fraudCheck: 'pending' | 'passed' | 'failed' | 'review';
    fraudScore?: number;           // 0-100
    riskLevel: 'low' | 'medium' | 'high';
    twoFactorRequired: boolean;
    twoFactorVerified?: boolean;
    deviceFingerprint?: string;
    ipAddresses: {
        sender: string;
        recipient: string;
    };
}

/**
 * Юридическая информация
 */
export interface PaymentLegalInfo {
    termsAccepted: boolean;
    termsVersion: string;
    privacyAccepted: boolean;
    kycRequired: boolean;
}

```

```

kycVerified?: boolean;
taxReporting: boolean;
jurisdiction: string;
regulatoryCompliance: string[];
}

```

2.2 Дополнительные типы

```

/**
 * Запрос на перевод
 */
export interface PaymentRequest {
  requestId: string;
  recipient: string;           // userId или phone
  amount: number;
  currency: PaymentCurrency;
  message?: string;
  expiresAt?: number;
  status: 'pending' | 'accepted' | 'declined' | 'expired';
  createdAt: number;
}

/**
 * Лимиты пользователя
 */
export interface PaymentLimits {
  daily: {
    limit: number;
    used: number;
    remaining: number;
    resetAt: number;
  };
  monthly: {
    limit: number;
    used: number;
    remaining: number;
    resetAt: number;
  };
  perTransaction: {
    min: number;
    max: number;
  };
  isSheikh: boolean;
  sheikhMultiplier: number;   // x5 для Шейх
}

/**
 * История транзакций
 */
export interface PaymentTransaction {

```

```

transactionId: string;
paymentId: string;
type: 'send' | 'receive' | 'refund' | 'fee';
amount: number;
currency: PaymentCurrency;
counterparty: PaymentUser;
status: PaymentStatus;
createdAt: number;
completedAt?: number;
}

/**
 * Настройки платежей
 */
export interface PaymentSettings {
  defaultCurrency: PaymentCurrency;
  defaultPaymentMethod: PaymentMethod;
  autoAcceptFromContacts: boolean;
  requireConfirmation: boolean;
  showReceipts: boolean;
  notifications: {
    onReceive: boolean;
    onSend: boolean;
    onStatusChange: boolean;
  };
};
}

```

3. API SPECIFICATION

3.1 Endpoints

PAYMENT API ENDPOINTS		
POST	/api/v1/payments/create	Создать перевод
POST	/api/v1/payments/:id/confirm	Подтвердить
POST	/api/v1/payments/:id/cancel	Отменить
POST	/api/v1/payments/:id/refund	Вернуть
GET	/api/v1/payments/:id	Получить
GET	/api/v1/payments/history	История
GET	/api/v1/payments/limits	Лимиты
POST	/api/v1/payments/request	Запросить
POST	/api/v1/payments/request/:id/respond	Ответить
GET	/api/v1/payments/receipt/:id	Получить чек

3.2 Создать перевод

Endpoint: `POST /api/v1/payments/create`

Headers:

```
Authorization: Bearer <token>
Content-Type: application/json
X-Sheikh-Status: true // Если пользователь Шейх
```

Request Body:

```
{
  "recipientId": "user_123456",
  "recipientPhone": "+79991234567", // Альтернативно
  "amount": 500000, // 5000.00 RUB в копейках
  "currency": "RUB",
  "paymentMethod": "sbp",
  "message": "Возвращаю долг за билет",
  "requireConfirmation": false,
  "metadata": {
    "chatId": "chat_789",
    "messageId": "msg_456"
  }
}
```

Response (Success 200):

```
{
  "success": true,
  "data": {
    "paymentId": "pay_abc123",
    "attachmentId": "att_xyz789",
    "status": "processing",
    "amount": 500000,
    "currency": "RUB",
    "fee": 0,
    "feePercent": 0,
    "sheikhDiscount": true,
    "estimatedCompletion": 1718400000000,
    "message": "Перевод создан и обрабатывается"
  }
}
```

Response (Premium Required 403):


```

{
  "success": false,
  "error": {
    "code": "PREMIUM_REQUIRED",
    "message": "Переводы доступны только пользователям с статусом Шейх",
    "upgradeUrl": "/api/v1/premium/upgrade",
    "pricing": {
      "monthly": 999,
      "yearly": 9999,
      "lifetime": 29999
    },
    "benefits": [
      "Переводы без комиссии",
      "Игры в чате",
      "Исчезающие сообщения",
      "Совместные документы",
      "Комбинированные вложения"
    ]
  }
}

```

Response (Insufficient Funds 402):

```

{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "Недостаточно средств на балансе",
    "available": 300000,
    "required": 500000,
    "shortfall": 200000,
    "topUpUrl": "/api/v1/payments/topup"
  }
}

```

Response (Limit Exceeded 429):

```

{
  "success": false,
  "error": {
    "code": "DAILY_LIMIT_EXCEEDED",
    "message": "Превышен дневной лимит переводов",
    "dailyLimit": 1000000,
    "usedToday": 950000,
    "requested": 500000,
    "resetAt": 1718486400000
  }
}

```

3.3 Подтвердить перевод

Endpoint: `POST /api/v1/payments/:id/confirm`

Request Body:

```
{
  "twoFactorCode": "123456", // Если требуется 2FA
  "confirmIdentity": true
}
```

Response:

```
{
  "success": true,
  "data": {
    "paymentId": "pay_abc123",
    "status": "completed",
    "completedAt": 1718400123456,
    "transactionId": "txn_def456",
    "receipt": {
      "receiptId": "rcp_789",
      "receiptUrl": "https://balloo.app/receipts/rcp_789"
    }
  }
}
```

3.4 История переводов

Endpoint: `GET /api/v1/payments/history`

Query Parameters:

```
?limit=50
&offset=0
&type=send,receive
&status=completed
&from=1717200000000
&to=1718400000000
&counterparty=user_123
```

Response:

```

{
  "success": true,
  "data": {
    "transactions": [
      {
        "transactionId": "txn_def456",
        "paymentId": "pay_abc123",
        "type": "send",
        "amount": 500000,
        "currency": "RUB",
        "counterparty": {
          "userId": "user_789",
          "displayName": "Иван Петров",
          "avatar": "https://..."
        },
        "message": "Возвращаю долг",
        "status": "completed",
        "createdAt": 1718400000000,
        "completedAt": 1718400123456
      }
    ],
    "pagination": {
      "total": 150,
      "limit": 50,
      "offset": 0,
      "hasMore": true
    },
    "summary": {
      "totalSent": 15000000,
      "totalReceived": 8000000,
      "totalFees": 0,
      "currency": "RUB"
    }
  }
}

```

3.5 Лимиты пользователя

Endpoint: GET /api/v1/payments/limits

Response:

```

{
  "success": true,
  "data": {
    "isSheikh": true,
    "sheikhType": "monthly",
    "sheikhExpiresAt": 1721078400000,
    "limits": {
      "daily": {

```

```

    "limit": 5000000,
    "used": 500000,
    "remaining": 4500000,
    "resetAt": 1718486400000
  },
  "monthly": {
    "limit": 50000000,
    "used": 12000000,
    "remaining": 38000000,
    "resetAt": 1720137600000
  },
  "perTransaction": {
    "min": 100,
    "max": 1000000
  }
},
"multipliers": {
  "sheikhBonus": 5,
  "verifiedBonus": 2,
  "totalMultiplier": 10
}
}
}

```

4. БИЗНЕС-ЛОГИКА

4.1 Проверка доступа

```

async function validatePaymentAccess(
  userId: string,
  amount: number,
  currency: PaymentCurrency
): Promise<ValidationResult> {

  // 1. Проверка статуса Шейх
  const user = await getUserById(userId);

  if (!user.is_sheikh) {
    throw new PremiumRequiredError('payment');
  }

  // 2. Проверка срока действия
  if (user.sheikh_expires_at && user.sheikh_expires_at < Date.now()) {
    throw new PremiumExpiredError();
  }

  // 3. Проверка лимитов
  const limits = await getPaymentLimits(userId);

```

```

if (amount > limits.daily.remaining) {
  throw new DailyLimitExceededError(limits);
}

if (amount > limits.perTransaction.max) {
  throw new TransactionLimitExceededError(limits);
}

// 4. Проверка баланса (если оплата с баланса)
if (paymentMethod === 'balance') {
  const balance = await getBalance(userId);
  if (balance < amount) {
    throw new InsufficientFundsError(balance, amount);
  }
}

// 5. KYC проверка для больших сумм
if (amount >= 1000000) { // 10000 RUB
  if (!user.kyc_verified) {
    throw new KycRequiredError();
  }
}

// 6. Fraud проверка
const fraudScore = await calculateFraudScore(userId, amount);
if (fraudScore > 80) {
  throw new FraudDetectedError(fraudScore);
}

return { allowed: true };
}

```

4.2 Расчёт комиссии

```

function calculatePaymentFee(
  amount: number,
  paymentMethod: PaymentMethod,
  isSheikh: boolean,
  currency: PaymentCurrency
): number {

  // Базовая комиссия по методу
  const baseRates: Record<PaymentMethod, number> = {
    balance: 0, // 0%
    sbp: 0, // 0% для СБП
    card: 1.5, // 1.5%
    yandex: 2.0, // 2.0%
    tinkoff: 1.8 // 1.8%
  };
}

```

```

let feePercent = baseRates[paymentMethod];

// Скидка для Шейх
if (isSheikh) {
    feePercent = 0; // Полное освобождение
}

// Конвертация валюты
if (currency !== 'RUB') {
    feePercent += 0.5; // +0.5% за конвертацию
}

// Минимальная комиссия
const minFee = 10; // 10 копеек
const calculatedFee = Math.round(amount * (feePercent / 100));

return Math.max(calculatedFee, minFee);
}

```

4.3 Обработка перевода

```

async function processPayment(paymentId: string): Promise<void> {
    const payment = await getPayment(paymentId);

    try {
        // 1. Статус: processing
        await updatePaymentStatus(paymentId, 'processing');

        // 2. Списываем средства
        await debitAmount(payment.sender.userId, payment.amount + payment.fee);

        // 3. Внешний платёж (если не баланс)
        if (payment.paymentMethod !== 'balance') {
            const externalResult = await processExternalPayment({
                method: payment.paymentMethod,
                amount: payment.amount,
                recipient: payment.recipient,
                reference: payment.reference
            });

            if (!externalResult.success) {
                throw new ExternalPaymentError(externalResult.error);
            }
        }

        // 4. Зачисляем получателю
        await creditAmount(payment.recipient.userId, payment.amount);

        // 5. Генерируем чек
        const receipt = await generateReceipt(payment);
    }
}

```

```

// 6. Статус: completed
await updatePaymentStatus(paymentId, 'completed', {
  completedAt: Date.now(),
  receiptId: receipt.receiptId
});

// 7. Уведомления
await sendNotification(payment.sender.userId, {
  type: 'payment_sent',
  paymentId: payment.id
});

await sendNotification(payment.recipient.userId, {
  type: 'payment_received',
  paymentId: payment.id
});

} catch (error) {
  // Откат при ошибке
  await rollbackPayment(paymentId, error);
  await updatePaymentStatus(paymentId, 'failed', {
    reason: error.message
  });
}
}

```

5. UI/UX SPECIFICATION

5.1 Интерфейс создания перевода

← Новый перевод

Кому:

Иван Петров

+7 999 123...

[Выбрать из контактов]

Сумма:

5 000.00 ₺

[₺ \$ €]

Метод оплаты:

о Баланс Balloo

(5 000.00 ₺)

- | | | |
|--------------------|--------------|-------------|
| ○ СБП | (5 000.00 ₺) | ⚡ Мгновенно |
| ○ Банковская карта | (5 075.00 ₺) | +1.5% |
| ○ Яндекс.Деньги | (5 100.00 ₺) | +2.0% |

Комментарий (необязательно):

Возвращаю долг за билет...

Комиссия: 0.00 ₺ (Скидка Шейх 100%)

К списанию: 5 000.00 ₺

[Отменить]

[Перевести 5 000.00 ₺]

5.2 Карточка перевода в чате

💰 Перевод

От: Иван Петров

Кому: Вы

5 000.00 ₺

Возвращаю долг за билет

☒ Перевод завершён

15 июня 2026, 14:30

[📄 Чек] [💬 Ответить] [🔗 Поделиться]

5.3 Модальное окно (не Шейх)

💎 Требуется статус Шейх

Переводы доступны только пользователям со статусом 👑 Шейх

Что вы получите:

- ☒ Переводы без комиссии
- ☒ Игры в чате
- ☒ Исчезающие сообщения
- ☒ Совместные документы
- ☒ Комбинированные вложения
- ☒ Файлы до 500 МВ
- ☒ Эксклюзивные стикеры

 \$9.99/мес  \$99.99/год  \$299.99 навсегда

[ Отмена] [ Оформить подписку]

6. БЕЗОПАСНОСТЬ

6.1 Требования безопасности

```
interface PaymentSecurityRequirements {  
  // Шифрование  
  encryption: 'AES-256-GCM';  
  tlsVersion: '1.3';  
  
  // Аутентификация  
  requireTwoFactor: boolean;           // Для сумм > 10000 RUB  
  twoFactorMethods: ['sms', 'totp', 'email'];  
  sessionTimeout: number;              // 300000 ms (5 мин)  
  
  // Fraud detection  
  fraudDetection: {  
    enabled: boolean;  
    maxScore: number;                  // 80  
    autoBlockThreshold: number;        // 95  
    reviewThreshold: number;           // 60  
  };  
  
  // Лимиты  
  rateLimiting: {  
    requestsPerMinute: number;         // 10  
    transactionsPerHour: number;        // 20  
    transactionsPerDay: number;         // 100  
  };  
};
```

```

// Аудит
audit: {
  logAllTransactions: boolean;
  logAccessAttempts: boolean;
  retentionDays: number;          // 365
};

// Compliance
compliance: {
  kycRequired: boolean;
  kycThreshold: number;           // 10000 RUB
  amlChecks: boolean;
  taxReporting: boolean;
};
}

```

6.2 Fraud Detection

```

async function calculateFraudScore(
  userId: string,
  amount: number,
  recipientId: string
): Promise<number> {

  let score = 0;

  // 1. Новая учётная запись
  const userAge = await getUserAge(userId);
  if (userAge < 7 * 24 * 60 * 60 * 1000) { // < 7 дней
    score += 20;
  }

  // 2. Необычно большая сумма
  const avgTransaction = await getAverageTransaction(userId);
  if (amount > avgTransaction * 5) {
    score += 25;
  }

  // 3. Первый перевод этому получателю
  const hasHistory = await hasTransactionHistory(userId, recipientId);
  if (!hasHistory) {
    score += 15;
  }

  // 4. Подозрительный IP
  const ipRisk = await checkIpRisk(getUserIP());
  if (ipRisk > 50) {
    score += 30;
  }
}

```

```
// 5. Несколько устройств
const deviceCount = await getDeviceCount(userId);
if (deviceCount > 5) {
    score += 10;
}

// 6. Ночное время
const hour = new Date().getHours();
if (hour >= 2 || hour <= 5) {
    score += 10;
}

return Math.min(score, 100);
}
```

7. ЮРИДИЧЕСКИЕ ТРЕБОВАНИЯ

7.1 Необходимые документы

ЮРИДИЧЕСКИЕ ДОКУМЕНТЫ	
	Пользовательское соглашение
	Политика конфиденциальности
	Договор оферты на переводы
	Политика AML/CFT
	Информация о комиссиях
	Порядок рассмотрения претензий

7.2 Лицензии

Регион	Лицензия	Статус
Россия	Лицензия ЦБ РФ на НПО	Требуется
Казахстан	Лицензия НБРК	Требуется
Беларусь	Лицензия НБРБ	Требуется
ЕС	EMI License	Требуется

7.3 KYC Требования

```

interface KycRequirements {
  // Уровень 1 (базовый)
  level1: {
    required: boolean;
    fields: ['phone', 'email'];
    transactionLimit: 100000; // 1000 RUB
  };

  // Уровень 2 (расширенный)
  level2: {
    required: boolean;
    fields: ['passport', 'selfie', 'address'];
    transactionLimit: 1000000; // 10000 RUB
  };

  // Уровень 3 (полный)
  level3: {
    required: boolean;
    fields: ['income_proof', 'source_of_funds'];
    transactionLimit: Infinity;
  };
}

```

8. ИНТЕГРАЦИИ

8.1 Платёжные провайдеры

```

interface PaymentProviders {
  // СБП (Система Быстрых Платежей)
  sbp: {
    provider: 'NSPK';
    apiVersion: '2.0';
    settlement: 'T+0';
    commission: 0;
  };

  // Яндекс.Деньги (ЮKassa)
  yandex: {
    provider: 'YooKassa';
    apiVersion: '3.0';
    settlement: 'T+1';
    commission: 2.0;
  };

  // Тинькофф Касса
  tinkoff: {
    provider: 'Tinkoff';
    apiVersion: '2.0';
  };
}

```

```

    settlement: 'T+1';
    commission: 1.8;
};

// CloudPayments
cloudpayments: {
    provider: 'CloudPayments';
    apiVersion: '1.0';
    settlement: 'T+1';
    commission: 1.5;
};
}

```

8.2 Внутренний баланс

```

interface BallooBalance {
    // Пополнение
    topUp: {
        methods: ['card', 'sbp', 'yandex', 'tinkoff'];
        minAmount: 100;
        maxAmount: 1000000;
        instant: boolean;
    };

    // Вывод
    withdrawal: {
        methods: ['card', 'sbp', 'yandex'];
        minAmount: 500;
        maxAmount: 100000;
        processingTime: '1-3 business days';
        fee: 0; // Для Шейх
    };

    // Конвертация
    conversion: {
        supportedCurrencies: ['RUB', 'USD', 'EUR', 'KZT'];
        spread: 0.5; // 0.5%
    };
}

```

9. МЕТРИКИ

9.1 KPI

```

interface PaymentMetrics {
    // Объём
    totalVolume: number; // Общий объём переводов
}

```

```

totalTransactions: number; // Количество транзакций
averageTransaction: number; // Средняя сумма

// Конверсия
initiationToCompletion: number; // % завершённых
abandonmentRate: number; // % отменённых

// Доходы
totalFees: number; // Собранные комиссии
revenuePerUser: number; // ARPU

// Безопасность
fraudRate: number; // % fraudulent transactions
chargebackRate: number; // % chargebacks

// UX
averageProcessingTime: number; // Среднее время обработки
successRate: number; // % успешных
}

```

10. ТЕСТИРОВАНИЕ

10.1 Тест-кейсы

```

describe('Payment Attachment', () => {

  // Позитивные тесты
  it('should create payment for Sheikh user', async () => {
    // ...
  });

  it('should process payment without commission for Sheikh', async () => {
    // ...
  });

  it('should complete payment within 5 seconds', async () => {
    // ...
  });

  // Негативные тесты
  it('should reject payment for non-Sheikh user', async () => {
    // ...
  });

  it('should reject payment with insufficient funds', async () => {
    // ...
  });

  it('should reject payment exceeding daily limit', async () => {

```

```
    // ...
  });

  // Безопасность
  it('should require 2FA for large amounts', async () => {
    // ...
  });

  it('should detect and block fraudulent payment', async () => {
    // ...
  });

  // Edge cases
  it('should handle currency conversion correctly', async () => {
    // ...
  });

  it('should rollback on external payment failure', async () => {
    // ...
  });
});
```

 **Статус документа:** Complete

 **Baloo - Проверни общение!**